

1. Примитивный модуль

Для начала напишем примитивный модуль task1.

```
1 #include <linux/init.h>
2 #include <linux/module.h>
3
4 MODULE_LICENSE("GPL");
5
6 static int __init hello_init(void){
7     pr_info("Hello, world!\n");
8     return 0;
9 }
10
11 static void __exit hello_exit(void){
12     pr_info("Goodbye, cruel world!\n");
13 }
14
15 module_init(hello_init);
16 module_exit(hello_exit);
```

Рисунок 1 — содержание модуля task1

Затем напишем makefile для сборки модуля.

```
1 obj-m += task1.o
2
3 all:
4     make -C /lib/modules/6.8.0-117-generic/build M=$(PWD) modules
5
6 clean:
7     make -C /lib/modules/6.8.0-117-generic/build M=$(PWD) clean
```

Рисунок 2 — содержание Makefile

Теперь соберём модуль task1 и убедимся, что в коде отсутствуют ошибки.

```
shrek@hp-laptop:~/linux_kernel/modules$ vim task1.c
shrek@hp-laptop:~/linux_kernel/modules$ make
make -C /lib/modules/6.8.0-117-generic/build M=/home/shrek/linux_kernel/modules modules
make[1]: вход в каталог «/usr/src/linux-headers-6.8.0-117-generic»
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] /home/shrek/linux_kernel/modules/task1.o
MODPOST /home/shrek/linux_kernel/modules/Module.symvers
CC [M] /home/shrek/linux_kernel/modules/task1.mod.o
LD [M] /home/shrek/linux_kernel/modules/task1.ko
BTF [M] /home/shrek/linux_kernel/modules/task1.ko
Skipping BTF generation for /home/shrek/linux_kernel/modules/task1.ko due to unavailability of vmlinux
make[1]: выход из каталога «/usr/src/linux-headers-6.8.0-117-generic»
```

Рисунок 3 — результат выполнения сборки модуля task1

После собраный модуль можно подгрузить.

```
shrek@hp-laptop:~/linux_kernel/modules$ sudo insmod task1.ko  
[sudo] пароль для shrek:  
shrek@hp-laptop:~/linux_kernel/modules$
```

Рисунок 4 — загрузка модуля task1

Если вывести последние сообщения ядра, то можно увидеть, что task1 успешно инициализировался. Так же при помощи команды lsmod можно увидеть task1 в перечне остальных модулей.

```
shrek@hp-laptop:~/linux_kernel/modules$ dmesg | tail -n1  
[ 112.855674] Hello, world!  
shrek@hp-laptop:~/linux_kernel/modules$ lsmod | grep task1  
task1                12288  0
```

Рисунок 5 — результаты выполнения команд dmesg и lsmod

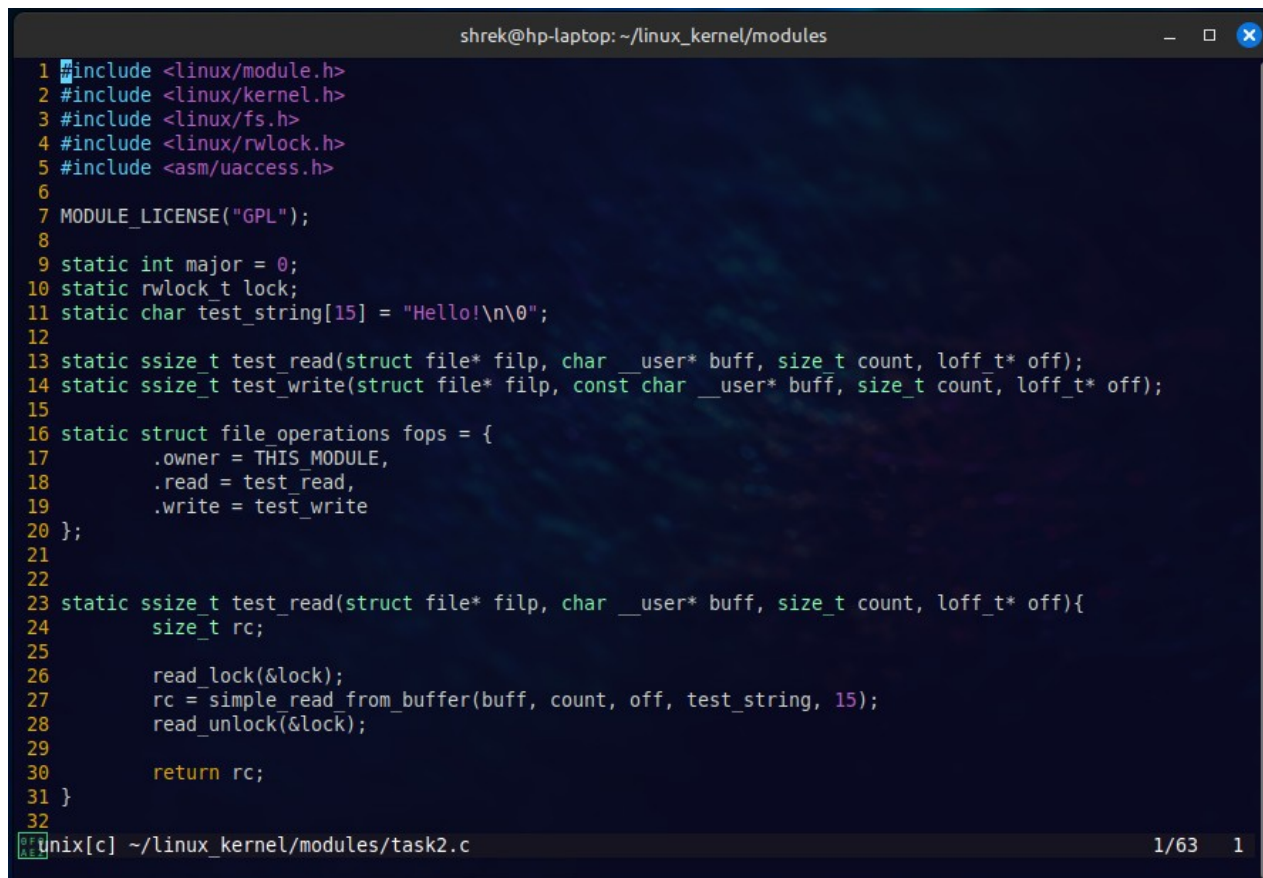
Теперь task1 можно выгрузить, снова вывести последние сообщения и убедиться, что модуль был успешно очищен. Так же если попытаться найти модуль в перечне других модулей, то его там не будет.

```
shrek@hp-laptop:~/linux_kernel/modules$ sudo rmmod task1  
shrek@hp-laptop:~/linux_kernel/modules$ dmesg | tail -n1  
[ 355.420077] Goodbye, cruel world!  
shrek@hp-laptop:~/linux_kernel/modules$ lsmod | grep task1  
shrek@hp-laptop:~/linux_kernel/modules$
```

Рисунок 6 — очистка модуля task1

2. Модуль с регистрацией файлового устройства

Для необходимо написать модуль task2.



```
shrek@hp-laptop: ~/linux_kernel/modules  
1 #include <linux/module.h>  
2 #include <linux/kernel.h>  
3 #include <linux/fs.h>  
4 #include <linux/rwlock.h>  
5 #include <asm/uaccess.h>  
6  
7 MODULE_LICENSE("GPL");  
8  
9 static int major = 0;  
10 static rwlock_t lock;  
11 static char test_string[15] = "Hello!\n\0";  
12  
13 static ssize_t test_read(struct file* filp, char __user* buff, size_t count, loff_t* off);  
14 static ssize_t test_write(struct file* filp, const char __user* buff, size_t count, loff_t* off);  
15  
16 static struct file_operations fops = {  
17     .owner = THIS_MODULE,  
18     .read = test_read,  
19     .write = test_write  
20 };  
21  
22  
23 static ssize_t test_read(struct file* filp, char __user* buff, size_t count, loff_t* off){  
24     size_t rc;  
25  
26     read_lock(&lock);  
27     rc = simple_read_from_buffer(buff, count, off, test_string, 15);  
28     read_unlock(&lock);  
29  
30     return rc;  
31 }  
32  
unix[c] ~/linux_kernel/modules/task2.c 1/63 1
```



```
shrek@hp-laptop: ~/linux_kernel/modules
32
33 static ssize_t test_write(struct file* filp, const char __user* buff, size_t count, loff_t* off){
34     size_t rc = 0;
35     if (count > 15) {
36         return -EINVAL;
37     }
38
39     write_lock(&lock);
40     rc = simple_write_to_buffer(test_string, 15, off, buff, count);
41     write_unlock(&lock);
42
43     return rc;
44 }
45
46
47 int init_module(void){
48     pr_info("Module is loaded\n");
49     rwlock_init(&lock);
50     major = register_chrdev(major, "task2", &fops);
51
52     if (major < 0) {
53         return major;
54     }
55     pr_info("Major = %d\n", major);
56
57     return 0;
58 }
59
60 void cleanup_module(void){
61     pr_info("Module is cleaned\n");
62     unregister_chrdev(major, "task2");
63 }
shrek@hp-laptop: ~/linux_kernel/modules/task2.c 63/63 1
```

Рисунок 7-8 — содержание модуля task2

Теперь собираем модуль с тем же makefile.

```
shrek@hp-laptop:~/linux_kernel/modules$ vim task2.c
shrek@hp-laptop:~/linux_kernel/modules$ make
make -C /lib/modules/6.8.0-117-generic/build M=/home/shrek/linux_kernel/modules modules
make[1]: вход в каталог «/usr/src/linux-headers-6.8.0-117-generic»
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] /home/shrek/linux_kernel/modules/task2.o
MODPOST /home/shrek/linux_kernel/modules/Module.symvers
CC [M] /home/shrek/linux_kernel/modules/task2.mod.o
LD [M] /home/shrek/linux_kernel/modules/task2.ko
BTF [M] /home/shrek/linux_kernel/modules/task2.ko
Skipping BTF generation for /home/shrek/linux_kernel/modules/task2.ko due to unavailability of vmlinux
make[1]: выход из каталога «/usr/src/linux-headers-6.8.0-117-generic»
```

Рисунок 9 — результат выполнения сборки task2

Загружаем модуль и выводим последние сообщения ядра. За одно получаем мажорный номер устройства.

```
shrek@hp-laptop:~/linux_kernel/modules$ sudo insmod task2.ko
[sudo] пароль для shrek:
shrek@hp-laptop:~/linux_kernel/modules$ dmesg | tail -n2
[ 445.613492] Module is loaded
[ 445.613501] Major = 237
shrek@hp-laptop:~/linux_kernel/modules$
```

Рисунок 10 — загрузка модуля task2

Затем создаём файл с таким же мажорным номером.

```
shrek@hp-laptop:/dev$ sudo mknod task2 c 237 0
[sudo] пароль для shrek:
shrek@hp-laptop:/dev$ ls -ls task2
0 crw-r--r-- 1 root root 237, 0 июн 14 13:08 task2
shrek@hp-laptop:/dev$
```

Рисунок 11 — создание файла task2

Теперь выводим содержимое данного файла при помощи команды head и убеждаемся, что модуль успешно считал ранее созданную переменную test_string.

```
shrek@hp-laptop:/dev$ head task2
Hello!
```

Рисунок 12 — вывод переменной test_string

Если попытаться перезаписать переменную при помощи команды echo и снова вывести её содержимое, то можно увидеть, что переменная действительно перезаписалась.

```
shrek@hp-laptop:/dev$ echo "ABCDEFGH" | sudo tee task2 > /dev/null
shrek@hp-laptop:/dev$ head task2
ABCDEFGH
```

Рисунок 13 — изменение переменной test_string

Затем можно выгрузить модуль, вывести последние сообщения и убедиться, что модуль очистился успешно.

```
shrek@hp-laptop:~/linux_kernel/modules$ sudo rmmod task2
shrek@hp-laptop:~/linux_kernel/modules$ dmesg | tail -n2
[ 915.448078] Major = 237
[ 923.700079] Module is cleaned
```

Рисунок 14 — очистка модуля task2